EfficientAD: What We Did and Learned

Session summary · Project: EfficientAD anomaly detection · Branch: `efficient_ad_2.0`

1. Goal

Make the model find defective images reliably. The priority is that a bad image must not be called good. False alarms are acceptable. The longer-term goal is a thesis contribution on fabric anomaly detection using conformal risk control.

2. How the model works (high level)

- EfficientAD is an **anomaly detector**. It trains on **good images only** and never sees a defect during training.
- A frozen **teacher** network gives features. A **student** network learns to copy the teacher on good images, and an **autoencoder** learns to reconstruct them.
- On a defective image the student cannot copy the teacher well, so the error is high at the defect. That error becomes a heatmap.
- The **image score is the single hottest pixel** of the heatmap. A **threshold** decides: score at or above the threshold means "bad".
- The threshold is the one setting that trades missed defects against false alarms.

3. Code changes made (in efficientad.py)

Feature	What it does	Status
`--auto_threshold` / `full_recall`	Sets the threshold to the lowest defect score, so recall is 100% with the fewest false alarms possible.	Ran on Kaggle
False-alarm table	Shows false alarms at recall targets of 100, 99, 98, 95 and 90%.	Ran on Kaggle
Score modes `max`, `topk`, `smooth`	Three ways to turn the heatmap into one image score, with a comparison table.	Ran on Kaggle
`--threshold_mode` (`best_f1` default)	Picks the threshold by best F1 or accuracy, or uses a fixed value. Prints accuracy, recall, specificity, precision, F1 and TN/FP/FN/TP.	`best_f1` ran on Kaggle
`--load_dir`	Reloads a saved model and only evaluates, skipping training.	Not run
`max_recall` with `--min_accuracy`	Highest recall among thresholds whose accuracy is at least the floor.	Not run
Threshold table	Shows the results at several thresholds, with the chosen one marked.	Not run
`target_recall` with `--target_recall`	Catches at least the chosen share of bad images and prints a defect-focused summary, with the number of good images flagged as the cost.	Not run

Everything compiles. "Not run" means it has not been executed with PyTorch, because the local machine has no `torch`. The thresholds in every mode are chosen on the test set, so reported recall is optimistic.

4. Results so far (real runs on Kaggle)

Data: the 15 MVTec categories merged into one dataset. Test set: 4,969 good and 687 defective images (5,656 total).

Run	AUC	Result
1 training step (pipeline test)	72.1	Meaningless. The model was untrained.
5,000 steps, 100% recall rule	89.5	98.1% of good images falsely flagged.
70,000 steps, 100% recall rule	89.4	99.0% falsely flagged. More training did not help.
5,000 steps, `best_f1` threshold (0.1393)	89.3	See the table below.

Latest run at threshold 0.1393

Metric	Value	Meaning
Accuracy	0.9282	5,250 of 5,656 correct
Recall	0.6070	417 of 687 defects caught
Specificity	0.9726	4,833 of 4,969 good images passed
Precision	0.7541	3 in 4 alerts are real defects
F1	0.6726	Balance of precision and recall
Confusion matrix	TN 4833, FP 136, FN 270, TP 417	270 defects missed

False alarms at a target recall (latest run)

Target recall	Threshold	Good images flagged
100%	0.0215	4,719 / 4,969 (95.0%)
99%	0.0314	4,407 / 4,969 (88.7%)
98%	0.0399	4,055 / 4,969 (81.6%)
95%	0.0518	3,191 / 4,969 (64.2%)
90%	0.0640	2,125 / 4,969 (42.8%)

Score-mode comparison

Mode	AUC	100%	99%	95%	90%
max	89.30	95.0%	88.7%	64.2%	42.8%
topk	89.17	95.7%	87.7%	62.7%	41.2%
smooth	89.01	95.7%	86.2%	61.1%	44.9%

The columns show the false-alarm rate at each recall target. The three scores are nearly identical, so keep `max`.

5. What we learned

- **The model is moderate, not usable yet.** AUC is about 89. It catches about 6 in 10 defects with a low false-alarm rate, or nearly all defects while flagging almost everything.

- **Training longer did not help.** AUC was 89.5 at 5,000 steps and 89.4 at 70,000, even though the loss fell from 3.3 to 1.25.
- **Accuracy flatters the model.** 88% of test images are good, so always saying "good" would score 87.9%. Look at recall and the number of missed defects (270).
- **Likely cause of the weakness, not proven:** 15 mixed categories share one model, one normalization and one threshold. Score scales differ between object types. You said this MVTec set is only a test, so these results show the pipeline works, not how the model will do on your real images.
- **The image score type does not matter here.** `max`, `topk` and `smooth` gave about the same AUC.
- **A better threshold alone cannot fix recall.** Improving recall without losing accuracy needs better score separation, which comes from the model or the data.

6. Kaggle workflow and problems we hit

- **Two code folders.** The notebook pulled into an inner clone but ran training in an older outer copy. Use one folder and check `git log --oneline -1` before running.
- **Wrong notebook CSV.** The manual cell rebuilt results from TIFF maps. Same-named images overwrite each other there, so it covered only 404 of 5,656 images. The script's own `image_scores.csv` is correct.
- **Output folder must be new.** The script uses `os.makedirs` and crashes if the folder exists. Use a new name each run.
- **Kaggle wipes `/kaggle/working`** when a session restarts. The saved model in `Model_11` was lost. Use Save Version or download the `trainings` folder.
- **Timing.** 5,000 steps take about 20 minutes in total. 70,000 steps took about 3 hours 2 minutes of training plus about 12 minutes of setup and evaluation.

```
%cd /kaggle/working/EfficientAD
!git fetch origin
!git checkout efficient_ad_2.0
!git pull origin efficient_ad_2.0
!python efficientad.py --dataset mvtec_ad --subdataset MVTec --model_size small \
    --weights models/teacher_small.pth \
    --mvtec_ad_path /kaggle/input/datasets/istiakahmednabil/mvtecc \
    --output_dir /kaggle/working/Model_NEW --train_steps 5000
```

7. Git and GitHub state

Branch	Commit	On GitHub?
`main`	a3dd411 (auto threshold)	Yes
`EfficientAD-1.1`	005db85 (score modes, load_dir, threshold modes)	Yes
`efficient_ad_2.0`	d4373f8 (max_recall, threshold table), plus the `target_recall` mode uncommitted	**No, local only**

> **Push problem.** Terminal pushes fail with a 403 because this machine's saved Git login is the account `tiffin6060-ops`, which has no write access to `IstiakNabil/EfficientAD`. Pushing from VS Code works. To fix the terminal, remove the `git:https://github.com` entries in Windows Credential Manager and sign in again as IstiakNabil.

GitHub branch names cannot contain spaces, so "EfficientAD 1.1" became `EfficientAD-1.1`. The `target_recall` change still has to be committed.

8. Conformal risk control (thesis idea)

The problem it solves

Today the threshold is chosen on the same photos used for testing, like checking the answers before an exam. The reported recall is optimistic and there is no guarantee about missed defects on new images.

The idea

- Set aside some defective images only for choosing the threshold (the calibration set). Test on the rest.
- For defect scores sorted from lowest to highest, use the **floor(alpha × (n+1))-th lowest score** as the threshold, where n is the number of calibration defects.
- Result: on new defective images the system misses at most alpha of them on average. The guarantee holds whatever the model quality, and the price is false alarms.

What it does and does not do

Question	Answer
Improves model accuracy or AUC?	No. The model is unchanged.
Makes the model faster?	No.
Makes the miss rate on unseen images predictable and guaranteed?	Yes. This is its value.
Controls false alarms?	No. They are the cost and must be reported.

Conditions

- Calibration and deployment images must be similar (same fabric, defect types, camera). A new fabric type breaks the guarantee.
- You need enough defective images: at least 19 for alpha = 5%.
- The guarantee is on average over calibration draws, not for every single draw.

Is it a strong contribution?

> **On its own, modest.** Conformal risk control and conformal anomaly detection already exist, so plain use on EfficientAD is an application and evaluation. For image-level scores it reduces to a quantile rule. Literature has not been searched yet, so check before claiming novelty.

What makes it stronger:

- Pixel-level risk: control the share of defect pixels missed using the `ground_truth` masks (best candidate for genuine novelty).
- Fabric-specific study: new fabric types, per-fabric or per-defect-type calibration, and showing where the guarantee fails.
- Rigorous evaluation: 200 to 1,000 random splits, empirical miss rate versus alpha, false-alarm cost, effect of calibration size, and a comparison with the current `best_f1` baseline.

Status: conformal risk control is **not implemented** yet. Only the baselines exist. Impact claims in the thesis must come from your own experiments.

9. Dataset for testing

- Suggested: **one MVTec AD category** (for example `bottle`). It has `train/good` plus `test/good` and defect folders, which the script reads directly.
- The license is CC BY-NC-SA 4.0, so non-commercial use only.
- The file size of the `bottle` download was not confirmed. A comparable category (capsule) was listed at 404 MB.
- For the thesis you need a **fabric dataset** with enough labelled defects (and masks for the pixel-level idea). None has been chosen yet.

10. Next steps

1. Commit the `target_recall` change and publish `efficient_ad_2.0` from VS Code.
2. Train and test on a single clean category to see how the model performs without the mixed-category problem.
3. Pick a fabric dataset and search the literature on conformal anomaly detection for industrial inspection.
4. Implement the conformal mode: calibration/test split, the quantile rule, and the repeated-split experiment.
5. Ask your supervisor whether an application-plus-evaluation contribution is enough, or whether the pixel-level and distribution-shift parts are needed.